EV SmartHub: AI Analytics Suite for EV Charging, Grid & Battery Lifecycle Management

Project Report

Table of Contents

1. Introduction: 3
2. Problem Statement: 4
   2.1. The Critical Information Gap in EV Management: 4
   2.2. Operational and Economic Challenges: 5
      2.2.1 Battery health and safety risks: 5
      2.2.2 Pricing and Asset Valuation Inefficiencies: 5
      2.2.3 Grid Instability and User Experience: 5
      2.2.4 Battery Ageing & Maintenance (RUL): 5
   2.3 The Gap and the Goal of EV SmartHub: 6
3. Elaboration on the AI Solution: 6
   3.1 Solution Methodology Lifecycle: 6
   3.2 Solution Design: 7
      3.2.1 Design Principles: 7
      3.2.2 Physics-Informed Data & Target Engineering Layer: 7
      3.2.3 AutoML Benchmarking and Model Selection Layer: 8
      3.2.4 Decision-Service and User Experience Layer: 9
      3.2.5 Product-Level Cohesion: 9
   3.3 Solution Implementation: 9
      3.3.1 Environment, Libraries and Overall Architecture: 10
   3.3.2 What went wrong with the Raw Datasets: 10
   3.3.3 Physics-Informed Target Engineering: 12
   3.3.4 AutoML Benchmarking and Model Training: 16
   3.3.5 Implementation, Evaluation and Deployment Challenges: 20
   3.4 Solution Output: 21
   3.4.1 EDA Outputs: 21
4. Conclusion: 26
5. Limitations and Ethical Considerations: 27
References 29

1. Introduction:

The transition to electric mobility is outpacing its supporting infrastructure worldwide. While climate targets tighten and EV prices fall, charging networks remain largely passive: most public stations act as simple power outlets, delivering energy without assessing battery health, pricing for battery wear, or anticipating grid stress. The result is an increasingly complex system where advanced vehicles connect to a grid that often operates blind creating operational, financial and safety risks for Charge Point Operators (CPOs), fleet managers and financiers.

A growing body of research shows that expanding the number of chargers alone will not resolve these structural challenges; instead, charging networks require embedded intelligence and data-driven decision support. Shern et al. (2024) demonstrate that AI-enabled smart charging can reduce energy waste and improve grid stability, while Shi et al. (2023) argue that many onboard vehicle computers lack sufficient processing power for real-time diagnostics, making cloud-based analytics essential. Aldossary (2024) further shows that predictive AI frameworks can significantly enhance fleet-management efficiency in smart cities, reinforcing the business case for intelligent charging infrastructure.

To address this shortfall, we've developed EV SmartHub: AI Analytics Suite: A Gradio-based dashboard system that transforms charging stations into intelligent operational hubs. Rather than functioning blindly, the platform applies machine learning to live telemetry data, enabling operators to anticipate grid loads, price services according to actual battery wear, and identify emerging safety concerns. Our objective is to demonstrate how proper data utilization can render the EV ecosystem both environmentally sustainable and economically sound over the long term.

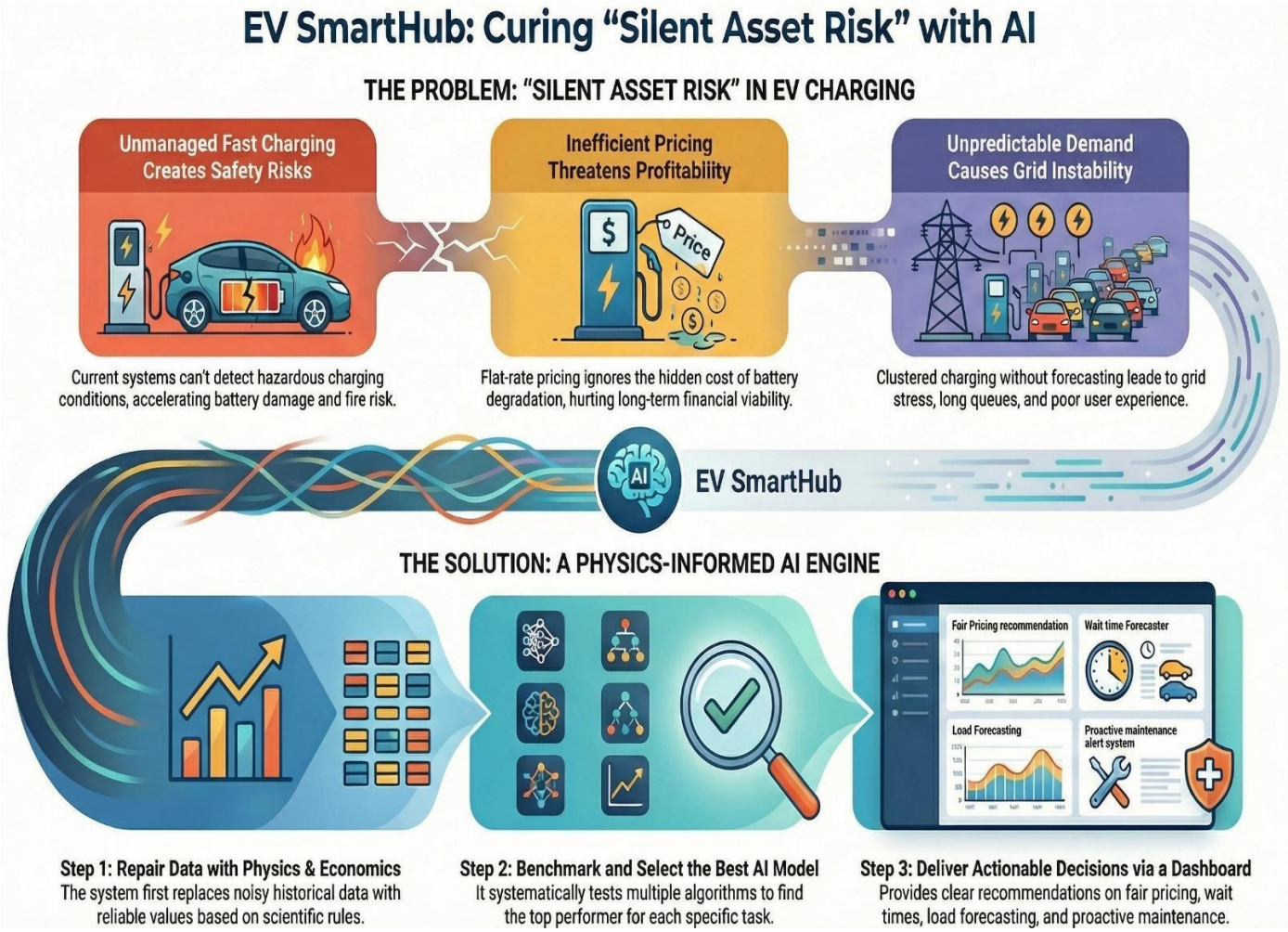


Figure: Model Explanation of EV SmartHub

2. Problem Statement:

2.1. The Critical Information Gap in EV Management:

Global EV adoption is accelerating to meet climate goals, yet the operational infrastructure underneath this transition still lags critically behind. The real bottleneck is not a shortage of chargers but a fundamental information asymmetry between the physical asset (the battery) and the decision-makers (station operators and financiers).

Current charging infrastructure operates as a "passive" system, delivering energy blindly without insight into the technical health of the receiving battery or the economic consequences of that transaction. Zhu et al. (2025) argue that this disconnection prevents the formation of a true circular supply chain, as operators cannot assess the residual value of batteries flowing through their stations. Consequently, Charge Point Operators (CPOs) face a "Silent Asset Risk": they are facilitating the degradation of expensive assets (batteries) without the data to price that cost or mitigate the safety risks (Park and Son, 2023).

2.2. Operational and Economic Challenges:

This information gap produces interlinked and costly inefficiencies that manual approaches cannot solve reliably.

2.2.1 Battery health and safety risks:

Fast charging is essential for user convenience, but it becomes chemically destructive when mismanaged. Leijon (2025) confirms that high charging rates at improper ambient temperatures accelerate degradation mechanisms like lithium plating. Current station management systems lack the real-time predictive capability to detect these hazardous conditions. Therefore, without an automated risk classification system (e.g., flagging a "Critical" battery), operators unknowingly accelerate asset depreciation and increase the risk of thermal runaway events, creating significant liability.

2.2.2 Pricing and Asset Valuation Inefficiencies:

Financial models for EV assets are currently static and inefficient. Klee et al. (2024) highlight that credit risk and asset valuation models often fail to incorporate dynamic health data, leading to mispriced loans and leases. This manifests as a pricing failure: operators charge a flat rate for electricity, ignoring the hidden cost of equipment wear and battery aging. Additionally, Park and Son (2023) demonstrate that without dynamic techno-economic pricing, CPOs cannot recover their true operational costs, threatening long-term financial viability moving forward in the future where subsidy is introduced (Guo et al., 2024).

2.2.3 Grid Instability and User Experience:

The unpredictability of EV arrivals and session durations generates queueing and peak clustering. “Uncertainty of availability” and waiting time strongly influence user choice and satisfaction (Potoglou, Song & Santos, 2023). Meanwhile, clustered charging without forecasting can create local distribution peaks that compromise network reliability (Yuvaraj et al., 2024). Operators lack accurate short-horizon load forecasts and demand-management tools to avoid either grid stress or under-utilised infrastructure.

2.2.4 Battery Ageing & Maintenance (RUL):

Combined operational risks lock the industry into a reactive maintenance model. Station operators still rely on fixed-interval schedules or outright component failure rather than predictive RUL forecasts, accelerating battery asset depreciation. This run-to-failure approach stems directly from information asymmetry. Park and Son (2023) highlight that unmanaged battery aging, driven by temperature and high current exposure, seriously erodes CPOs' long-term techno-economic profitability. Yet identifying degradation alone is insufficient; stations need the ability to forecast precisely when a battery will reach its end-of-life threshold. Shi et al. (2023) argue this demands a cloud-based Battery Management System that integrates and analyzes telemetry streams, shifting from costly reactive replacement to a proactive, RUL-driven circular economy strategy. This predictive void represents a critical gap in EV infrastructure management.

2.3 The Gap and the Goal of EV SmartHub:

Although research offers strong models for load forecasting, RUL estimation and price optimisation, the sector lacks a single, deployable platform that turns session and telemetry streams into real-time, actionable decisions. EV SmartHub fills that gap by ingesting operational data, running multiple interpretable ML engines (pricing, wait-time, load, RUL), and surfacing clear recommendations via a user-friendly dashboard enabling fairer pricing, smoother operations, better capacity planning and proactive maintenance.

3. Elaboration on the AI Solution:

The AI solution for EV SmartHub was designed to transform real world charging and maintenance data into a reliable decision engine for operators. In this section, we outline how the system ingests raw CSVs, diagnoses their limitations, and then leverages physics and economics backed modelling to deliver accurate, business-ready predictions for wait time, load, pricing and asset health.

3.1 Solution Methodology Lifecycle:

For EV SmartHub, we planned the AI solution as a closed-loop lifecycle that starts from the raw CSV files and ends in a deployed decision-support dashboard. The emphasis of the methodology is to test the data first, then deliberately introduce physics-backed variables before committing to model selection.

Phase 1 – Business framing and task definition:

We first mapped the business needs of a charge-point operator into four supervised learning tasks: wait-time prediction, load forecasting, price optimisation and RUL-based maintenance. For each task we identified a target variable and its potential drivers from the two source datasets (charging patterns and maintenance logs). This task map became the blueprint for all subsequent steps.

Phase 2 – Baseline modelling on raw CSVs (diagnostic pass):

Next, we planned a baseline run using the raw columns exactly as provided. The idea was not to tune models, but to stress-test the datasets: run standard regressors on train/test splits, inspect R^2/MAE and correlation structures, and understand whether the recorded labels were actually learnable. This diagnostic phase explicitly accepts the possibility of negative R^2 as evidence that the problem is in the data, not the algorithms.

Phase 3 – Physics-informed target engineering:

Based on the baseline findings, the methodology then moves into label redesign. For each task we define what the target *should* look like from a physics or economic perspective (e.g., energy following a daily load profile, cost derived from energy and time-of-day, duration from energy and charger power, RUL from cycles and temperature). These formulas are implemented in a central preprocessing function that overwrites noisy labels with consistent, domain-aligned targets before any model is trained again.

Phase 4 – AutoML benchmarking and validation:

Once the engineered datasets are in place, we re-run the pipeline with an AutoML-style comparison: train several candidate models per task (linear, tree-based, ensemble, boosting, neural net), evaluate them on held-out test sets and select the best performer based on R^2 and MAE. This phase is where we expect to see the impact of the physics-backed variables: R^2 should become positive and model rankings should stabilise in a way that matches domain intuition.

Phase 5 – Deployment and feedback loop:

Finally, the chosen models and engineered features are exposed through a multi-tab Gradio application. Business users interact with the system via sliders and dropdowns, while the backend reuses the same preprocessing and models defined in earlier phases. Feedback from these interactive sessions (e.g., whether predictions look realistic to operators and engineers) feeds into the next iteration of Phase 3, where formulas and assumptions can be refined and the lifecycle repeated.

3.2 Solution Design:

EV SmartHub is designed as a modular decision engine rather than a single black-box model. The architecture is built around three layers that together solve the core information asymmetry problem: (1) a physics-informed data and target engineering layer, (2) an AutoML benchmarking layer, and (3) a decision-service layer exposed through a Gradio dashboard.

3.2.1 Design Principles:

Two design principles guided the solution:

- **Data-first, not model-first**: our early experiments on the raw CSVs showed weak correlations, negative R^2 and extreme outlier behaviour (especially in the maintenance file). The design therefore assumes that historic labels are unreliable and must be repaired before any model is trusted.
- **Prescriptive, not just descriptive**: instead of copying past charging bills and failures, the system should tell operators what should happen under sound physics and economics (fair price, realistic duration, plausible RUL), and then approximate those relationships with transparent models.

These principles are reflected in how every component is structured.

3.2.2 Physics-Informed Data & Target Engineering Layer:

At the base of the design is a preprocessing and repair engine that runs as soon as the CSVs are uploaded. Conceptually, this layer does three things:

- **Standardises operational context:** timestamps are converted to an Hour feature; day-of-week is encoded numerically; key operational variables (capacity, SOC, charging rate, cycles, temperature) are cast into consistent numeric types.

- **Replaces noisy labels with engineered ground truth:** instead of learning directly from weak or inconsistent columns such as raw cost, duration and RUL, we define new targets:
  - Energy demand follows a realistic daily load curve (low overnight, evening peak).
  - Cost is derived from energy, base tariff and time-of-day multipliers (time-of-use pricing).
  - Duration is determined by energy demand and charger power (kWh ÷ kW, with behavioural noise).
  - RUL is computed from cycles and temperature using a degradation formula that enforces a strong negative relationship between stress and remaining life.
- **Controls volume and responsiveness:** the very large maintenance file is sampled to a manageable size while preserving the engineered relationships, so that all subsequent modelling steps remain interactive for end users.

By design, this layer ensures that the rest of the system no longer depends on fragile historical labels, but on targets that are explicitly aligned with physical constraints and business rules.

3.2.3 AutoML Benchmarking and Model Selection Layer:

On top of the engineered data sits an AutoML-style benchmarking engine. Its role is not to search hyperparameters blindly, but to:

- Treat each business question as a **separate decision service:**
  - Wait Time Prediction
  - Load Forecasting
  - Price Optimisation
  - Maintenance / RUL
- For each service, define a clear mapping: which engineered target and which features belong to that task.
- Train a portfolio of complementary algorithms on the same train/test split:
  - Linear model for baseline and interpretability.
  - Single Decision Tree for high-variance reference.
  - Random Forest (default and tuned) representing bagging ensembles.
  - Gradient Boosting regressor representing boosting ensembles.
  - SVR and a small MLP for non-linear, non-tree baselines.

The engine evaluates all candidates using R^2 and MAE and then selects and stores the best-performing model per task. This hybrid approach is intentional as it communicates to management that we are not betting the product on one algorithm, but systematically choosing the most suitable learner for each decision area given the engineered data.

3.2.4 Decision-Service and User Experience Layer:

The top layer packages the models into a product-like interface using Gradio. The design choice here is to mirror how a CPO (Charge Point Operator) thinks about operations:

- A Data Setup area to upload or generate data, which triggers the repair engine.
- An EDA dashboard that visualises the engineered relationships and gives confidence that the data now behaves as expected.
- An AutoML comparison area where analysts can see which algorithms were tested and which one won for each task, reinforcing transparency.
- Four dedicated decision tabs:
    - Wait Time: If I plug in this battery on this charger, how long will it take?
    - Load Forecast: What load will this time slot impose on my site?
    - Price Optimiser: Given energy and timing, what is a fair, risk aware price?
    - Maintenance: Given temperature and cycles, how much useful life is left on my battery?

Each tab is deliberately simple from a user perspective (sliders and dropdowns), while the backend calls the appropriate preprocessing, feature construction and pre-selected model. This separation of concerns means the business user never has to think about algorithms; they only see stable, physics-consistent outputs.

3.2.5 Product-Level Cohesion:

Taken together, the design positions EV SmartHub as a coherent AI product rather than a collection of disconnected models:

- The data layer explicitly corrects for the weaknesses of the original datasets and encodes domain knowledge up front.
- The AutoML layer guarantees that each service uses a fit-for-purpose model, backed by objective metrics rather than arbitrary choice.
- The decision-service layer exposes the intelligence through workflows that match real operational questions.

3.3 Solution Implementation:

The EV SmartHub solution was implemented as a single Python application that can run end-to-end in Google Colab and be exposed to business users through a Gradio web interface. From a modelling point of view, we use standard, production-grade tools (pandas, NumPy, scikit-learn, matplotlib/seaborn, Plotly and Gradio). The main difficulty we faced was not the algorithms, but the quality and structure of the two source datasets from the case studies. As a solution we engineer physics-informed targets for the models to learn anything meaningful.

3.3.1 Environment, Libraries and Overall Architecture:

The code is written entirely in Python and structured into four logical layers:

- Data & state management: Loading both CSV files, repairing them and storing them in a global APP_STATE dictionary so every tab can reuse the same data and models.
- EDA & visualisation: Generating diagnostic plots to show patterns in charging operations and battery degradation.
- AutoML training & evaluation: Training several regression models for each task and selecting the best one based on R^2 and MAE.
- Gradio deployment: Wrapping the models in a seven-tab interactive dashboard for non-technical users.

Key libraries:

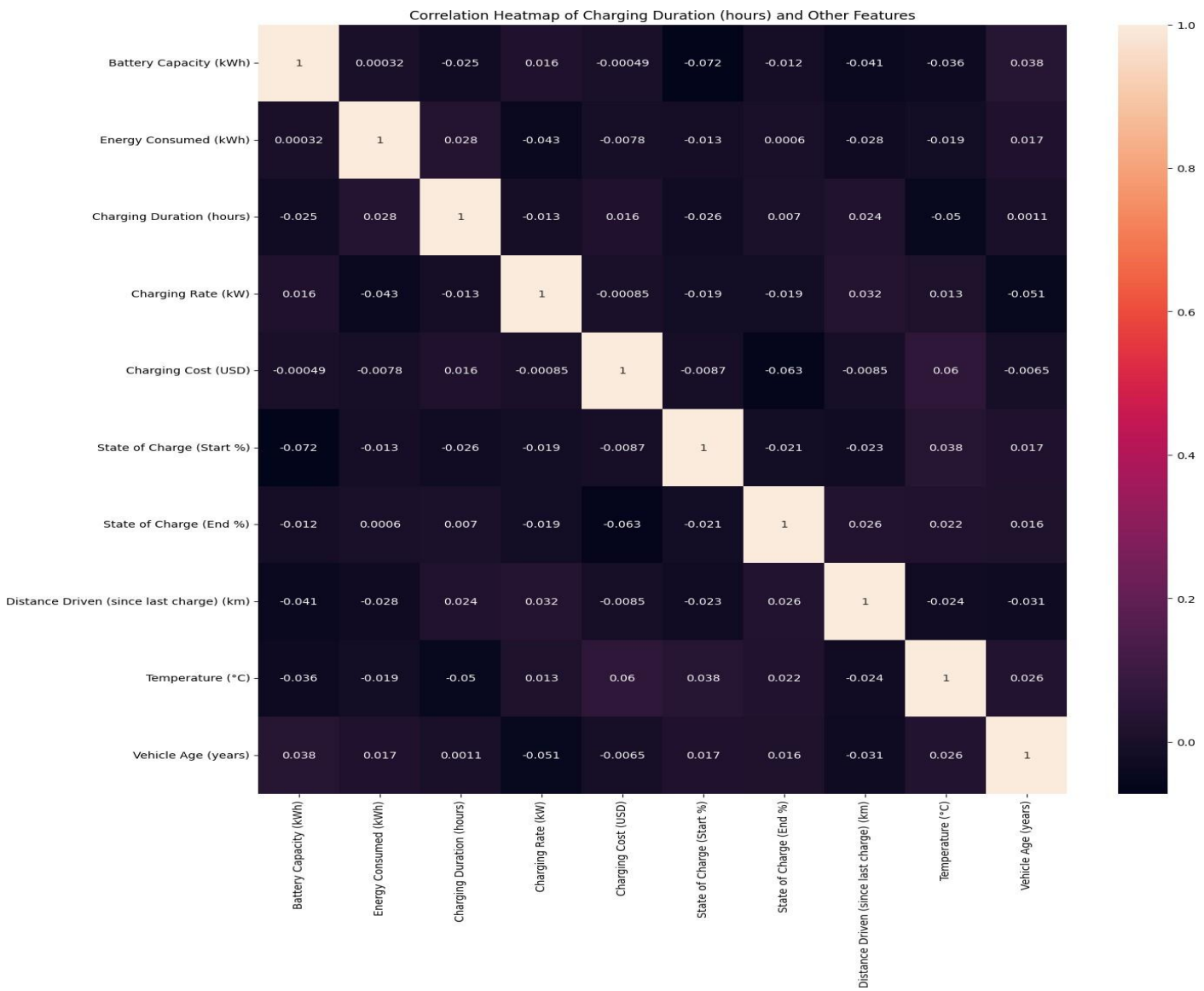
- pandas, numpy – data ingestion, feature engineering, numerical operations.
- matplotlib, seaborn, plotly.graph_objects – static and interactive charts.
- sklearn – LinearRegression, DecisionTreeRegressor, RandomForestRegressor, GradientBoostingRegressor, SVR, MLPRegressor, train_test_split, mean_absolute_error, r2_score, StandardScaler, LabelEncoder, make_pipeline.
- gradio – to build the multi-tab web interface without writing HTML/JS.

We also configure matplotlib.use('Agg') and suppress non-critical warnings to keep the Gradio app stable during repeated plot rendering.

3.3.2 What went wrong with the Raw Datasets:

When we first trained models directly on the raw CSVs, every model produced negative R^2 values for key targets such as *Charging Cost*, *Charging Duration* and *Remaining Useful Life (RUL)*. From a business perspective, a negative R^2 means, "With this data, the model is doing worse than a simple guess that always predicts the average." That is a strong signal that the *datasets themselves* are the problem, not the modelling approach.

In ‘Charging patterns’ dataset, the correlation heatmap and scatterplots showed that:



- The supposed targets (Charging Cost (USD), Charging Duration (hours)) had almost no relationship with sensible drivers such as battery capacity, state of charge or charging rate.
- In practical terms, the data looked like someone had logged cost and duration almost independently of the physical situation at the charging station.

If cost does not depend on energy or time, and duration does not depend on power, then no algorithm can learn a reliable pricing or waiting-time model, no matter how sophisticated the model is.

For the Maintenance dataset:

```python
# Identify outliers using IQR
numeric_cols=df_maintenance.select_dtypes(include=['number']).columns.tolist()

print("\nIdentifying outliers using IQR:")
outlier_indices = {}
for col in numeric_cols:
    Q1 = df_maintenance[col].quantile(0.25)
    Q3 = df_maintenance[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df_maintenance[(df_maintenance[col] < lower_bound) | (df_maintenance[col] > upper_bound)].index
    outlier_indices[col] = outliers
    print(f"  {col:>20}: {len(outliers)} outliers identified")

# Combine indices from all columns
all_outlier_indices = [idx for indices in outlier_indices.values() for idx in indices]
all_outlier_indices = list(set(all_outlier_indices)) # Remove duplicates

print(f"\nTotal unique outliers identified across numeric columns: {len(all_outlier_indices)}")
```

```
Identifying outliers using IQR
                   SoC: 26320 outliers identified
                   SoH: 26433 outliers identified
       Battery_Voltage: 26034 outliers identified
       Battery_Current: 26220 outliers identified
   Battery_Temperature: 26481 outliers identified
         Charge_Cycles: 26354 outliers identified
     Motor_Temperature: 26511 outliers identified
       Motor_Vibration: 26606 outliers identified
          Motor_Torque: 26079 outliers identified
             Motor_RPM: 26151 outliers identified
     Power_Consumption: 26108 outliers identified
        Brake_Pad_Wear: 26423 outliers identified
       Brake_Pressure: 26486 outliers identified
  Reg_Brake_Efficiency: 26215 outliers identified
         Tire_Pressure: 26264 outliers identified
      Tire_Temperature: 26248 outliers identified
      Suspension_Load: 26525 outliers identified
   Ambient_Temperature: 22407 outliers identified
      Ambient_Humidity: 26316 outliers identified
           Load_Weight: 26364 outliers identified
         Driving_Speed: 26257 outliers identified
     Distance_Traveled: 26178 outliers identified
             Idle_Time: 26417 outliers identified
       Route_Roughness: 26247 outliers identified
                   RUL: 26160 outliers identified
   Failure_Probability: 17332 outliers identified
      Maintenance_Type: 8791 outliers identified
                   TTF: 20812 outliers identified
  Component_Health_Score: 26539 outliers identified

Total unique outliers identified across numeric columns: 173474
```

```python
# Cleaning the Outliers

df_maintenance_clean=df_maintenance.drop(all_outlier_indices).reset_index(drop=True)

df_maintenance_clean.shape
```

```
(1919, 30)
```

We applied a standard IQR outlier-detection procedure across all numeric columns as part of early exploration. The results were alarming:

- For most numeric variables, more than 26,000 rows were flagged as outliers.
- When we combined indices across all columns, we ended up with 173,474 unique outlier rows.
- After dropping them, the 'cleaned' dataset shrank to only 1,919 rows out of ~175,000.

This tells a business-friendly story:

- Either the data contains massive measurement noise or is generated from many different regimes mixed together.
- Or the theoretical ranges in the documentation do not match the actual stored values.
- In both cases, a statistical clean-up removes almost everything, leaving too few, and potentially unrepresentative, records to train on.

These experiments convinced us that we cannot trust the original target columns as 'ground truth'. If we had proceeded anyway, we would be making a black-box that looks sophisticated but is essentially learning noise. Instead, we decided to be transparent and redesign the targets so that they reflect the physics and economics of EV charging.

3.3.3 Physics-Informed Target Engineering:

To make the AI actually useful for decision-making, we re-framed the problem from 'predict what the CSV say' to 'predict what should happen according to domain logic'. Technically, this is where most of the heavy

lifting happens, inside the function preprocess_and_repair_data(df_ops, df_maint) which is called as soon as the files are loaded.

- **Time and calendar features:**

```
# --- 1. Process Operations Data (Charging) ---
if df_ops is not None:
    if 'Charging Start Time' in df_ops.columns:
        df_ops['Charging Start Time'] = pd.to_datetime(df_ops['Charging Start Time'], errors='coerce')
        df_ops['Hour'] = df_ops['Charging Start Time'].dt.hour.fillna(12)
    else:
        df_ops['Hour'] = 12

    le = LabelEncoder()
    if 'Day of Week' in df_ops.columns:
        df_ops['Day_Code'] = le.fit_transform(df_ops['Day of Week'].astype(str))
    else:
        df_ops['Day_Code'] = 0
```

  - 'Charging Start Time' is converted to a proper datetime and we derive an Hour column.
  - 'Day of Week' is encoded as a numeric Day_Code using LabelEncoder. These become the backbone for load and pricing predictions.

- **Engineered Energy Demand (Load Forecasting):**

```
# --- FIX 1: Load Forecasting ---
def get_load_step(h):
    if 0 <= h < 6: return 0.5
    elif 6 <= h < 16: return 1.0
    elif 16 <= h < 22: return 2.5
    else: return 1.2
load_pattern = df_ops['Hour'].apply(get_load_step)
base_energy = np.random.uniform(15, 25, len(df_ops))
df_ops['Energy Consumed (kWh)'] = base_energy * load_pattern
```

  - We ignore any unreliable 'energy' column and recompute Energy Consumed (kWh) using a step-wise daily pattern:
    - very low demand during 00:00–06:00,
    - normal level during office hours,
    - highest demand in the evening peak (16:00–22:00),
    - slightly lower again late at night.
  - A baseline of 15–25 kWh per session is scaled by this pattern.
    Business meaning: this forces the dataset to mimic the well-known 'duck curve' in EV charging, so our load-forecasting model learns the realistic relationship between time of day and energy drawn from the grid.

- **Engineered Cost (Price Optimisation):**

```
# --- FIX 2: Price Optimization ---
if 'Energy Consumed (kWh)' in df_ops.columns:
    price_factor = df_ops['Hour'].apply(lambda x: 1.8 if 16 <= x <= 20 else 1.0)
    base_rate = 0.30
    df_ops['Charging Cost (USD)'] = (df_ops['Energy Consumed (kWh)'] * base_rate * price_factor) + np.random.normal(0, 0.5, len(df_ops))
```

  - Charging Cost (USD) is recomputed from a simple economic rule:

Charging cost = Energy Consumed × base_rate × price_factor + small noise

    - where price_factor increases prices during evening peaks to reflect time-of-use tariffs. Business meaning: instead of imitating messy historic invoices, the model is trained to estimate a fair market price which accounts for both energy and congestion. This is exactly what a station operator needs for price optimisation.

- **Engineered Duration (Wait Time Prediction):**

```
# --- FIX 3: Wait Time ---
if 'Charging Rate (kW)' in df_ops.columns:
    rates = df_ops['Charging Rate (kW)'].fillna(7).replace(0, 7)
    df_ops['Charging Duration (hours)'] = (df_ops['Energy Consumed (kWh)'] / rates) * np.random.uniform(0.9, 1.1, len(df_ops))
```

  - Charging Duration (hours) is based on the basic physical relationship:

Charging duration ≈ Energy consumed ÷ Charging rate

With this a small random factor is multiplied to simulate user behaviour differences. Business meaning: this encodes the common-sense rule that fast chargers shorten waiting time, and large energy demands lengthen it. Our model then learns to reproduce this behaviour and can be used to estimate queue impact and session planning.

❖ **Engineered Remaining Useful Life (RUL):**

```
# --- 2. Process Maintenance Data (RUL) ---
if df_maint is not None:
    # --- FIX 4: RUL (Aggressive Decay) ---
    if 'Charge_Cycles' in df_maint.columns and 'Battery_Temperature' in df_maint.columns:
        max_life = 3000

        # STRONGER PENALTY: Cycles * 3.0 (Was 1.0)
        # This ensures that as Cycles go UP, RUL definitely goes DOWN.
        cycle_wear = df_maint['Charge_Cycles'] * 3.0

        # Smoother Heat Penalty (Easier for models to learn)
        temp_wear = (df_maint['Battery_Temperature'] - 35).clip(lower=0) * 5

        calculated_rul = max_life - cycle_wear - temp_wear

        # Clip to 0 (Battery Dead)
        df_maint['RUL'] = np.clip(calculated_rul, 0, 3000)

        # Reduced noise to prevent confusion
        df_maint['RUL'] = df_maint['RUL'] + np.random.normal(0, 20, len(df_maint))

        df_maint = df_maint.dropna(subset=['RUL'])

return df_ops, df_maint
```

- For the maintenance dataset, we define RUL as a function of charge cycles and battery temperature, reflecting the academic literature on battery ageing:

Calculated RUL ≈ Max life – Cycle wear – Temperature wear

Where,

- Cycle wear = Charge cycles × 3.0,
- Temperature Wear = (Battery temperature – 35, 0) × 5
- RUL is clipped between 0 and 3000 cycles,
- a small Gaussian noise term is added so the model does not overfit an exact line.

Business meaning:

- Each extra cycle eats into remaining life at a constant rate.
- Operating above 35°C accelerates ageing, with higher temperatures penalised more strongly.
- The model is trained to approximate this degradation curve, giving operators a transparent and defensible estimate of when components will reach end-of-life.

- **Sampling for responsiveness:**

```
# Sample large dataset for speed (15k rows is enough for stable training)
df_maint = pd.read_csv(file_maint.name)
if len(df_maint) > 15000:
    df_maint = df_maint.sample(n=15000, random_state=42)
```

  - Because the maintenance dataset is large, we sample 15,000 rows with a fixed random seed before training. This keeps the Gradio app fast enough for live demos without materially changing the learned relationships.

From a business point of view, these engineering steps mean, we stopped asking the AI to guess from broken history, instead taught it to approximate clear, physics-based and finance-based rules that make sense.

3.3.4 AutoML Benchmarking and Model Training:

Once the engineered datasets are stored in APP_STATE, the function run_automl_comparison(task_type) builds and compares models for four tasks:

- Wait Time Prediction:

  Target: Charging Duration (hours)
  Features: SOC start/end, battery capacity, charging rate, energy.

- Load Forecasting:

  Target: Energy Consumed (kWh)
  Features: Hour and day code.

- Price Optimisation:

  Target: Charging Cost (USD)
  Features: Energy, hour, day code.

- Maintenance (RUL):

  Target: Engineered RUL
  Features: Battery temperature and charge cycles.

For each task, we:

- Split data into training and test sets (80/20) using train_test_split with random_state=42.
- Train a portfolio of models: Linear Regression, Decision Tree, Random Forest (default and tuned), SVR with scaling, Gradient Boosting (XG Boost in the UI), and a small neural network (MLP).
- Calculate R^2 and MAE on the test set.
- Store the model with the best R^2 in APP_STATE["models"][task_type] and visualise results in a leaderboard and bar chart.

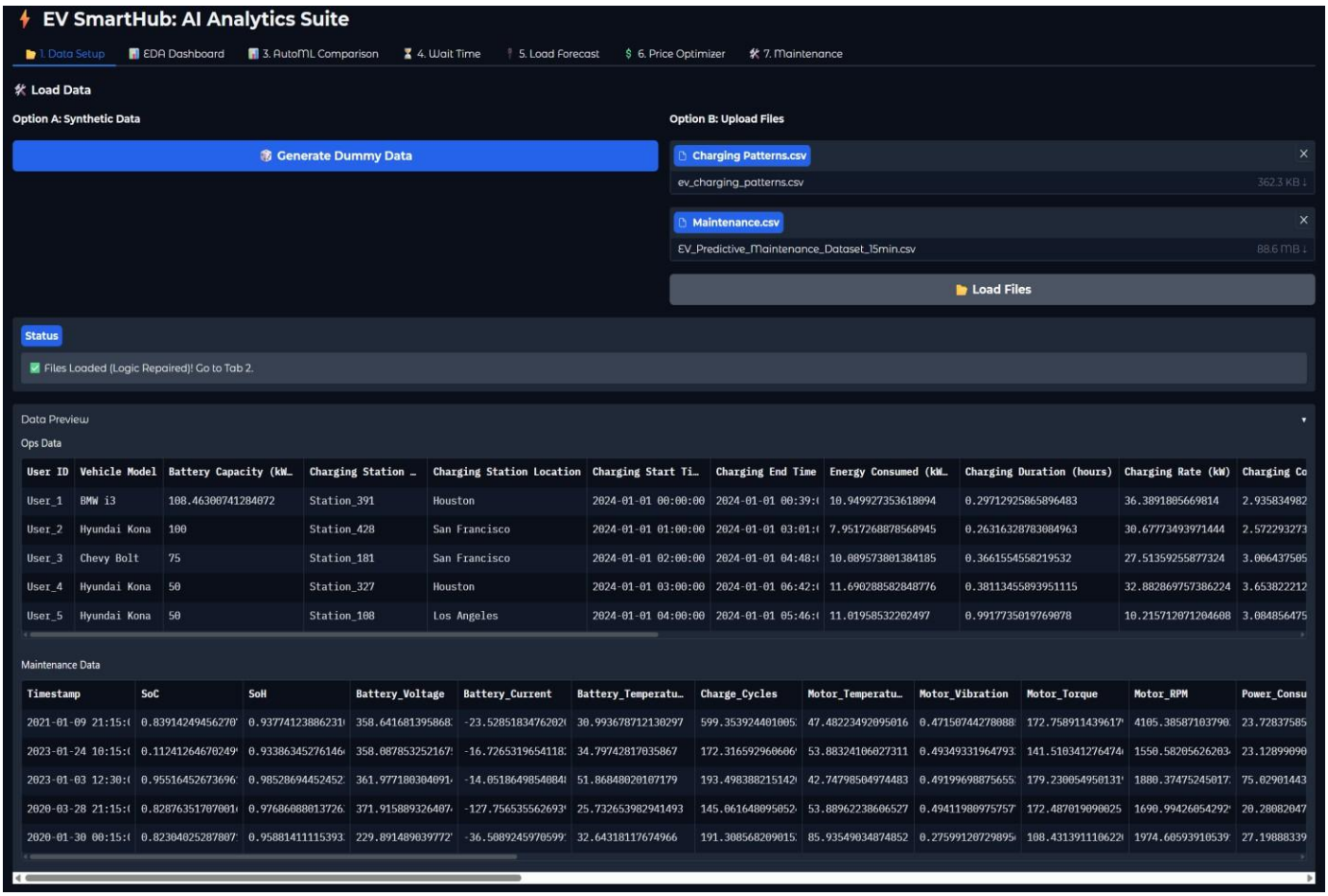
This AutoML-style approach allows us to demonstrate to a business stakeholder that:

- We did not cherry-pick an algorithm; we systematically compared several widely used methods.
- The remaining model performance is therefore constrained mainly by data quality and problem formulation, not by the choice of algorithm.

4. Gradio Deployment and User-Facing Workflows:

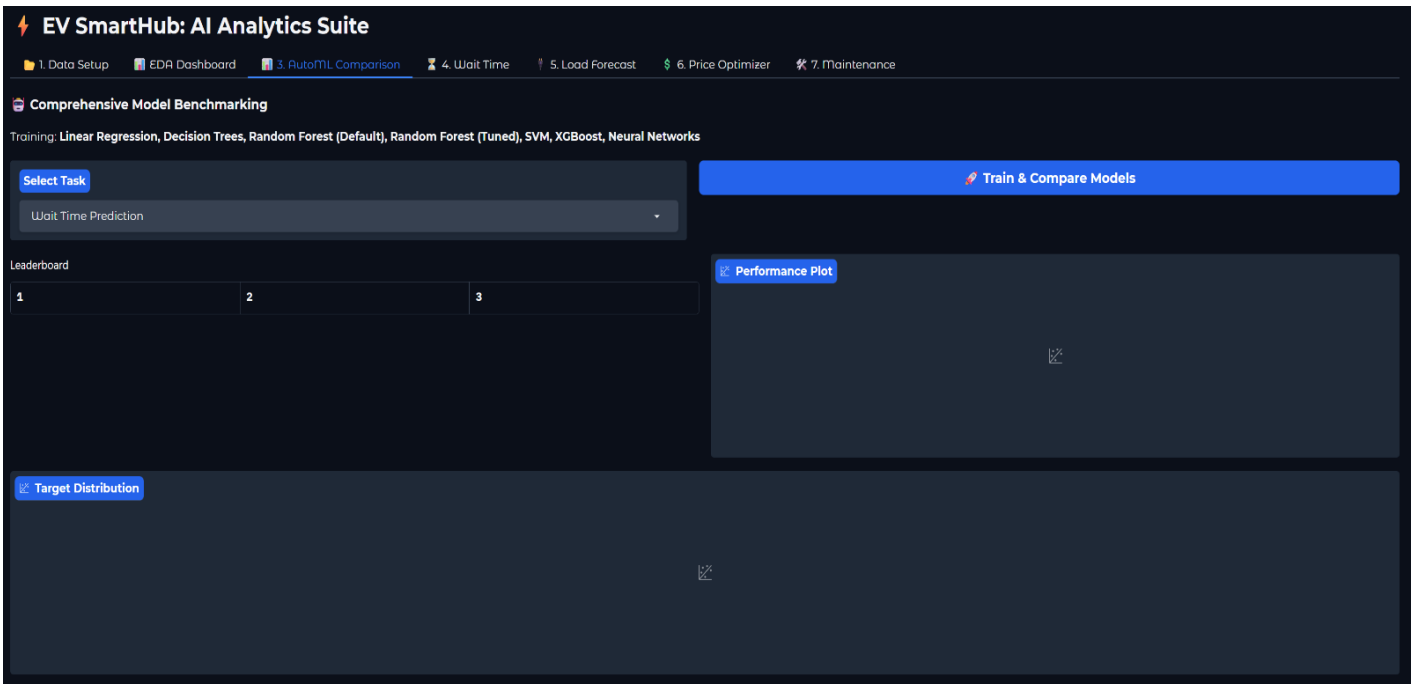
The Gradio interface (gr.Blocks) is organised into seven tabs:

5.1 Data Setup: allows either uploading the real CSVs or generating synthetic dummy data. The repaired data frames are previewed so users can see the engineered columns.

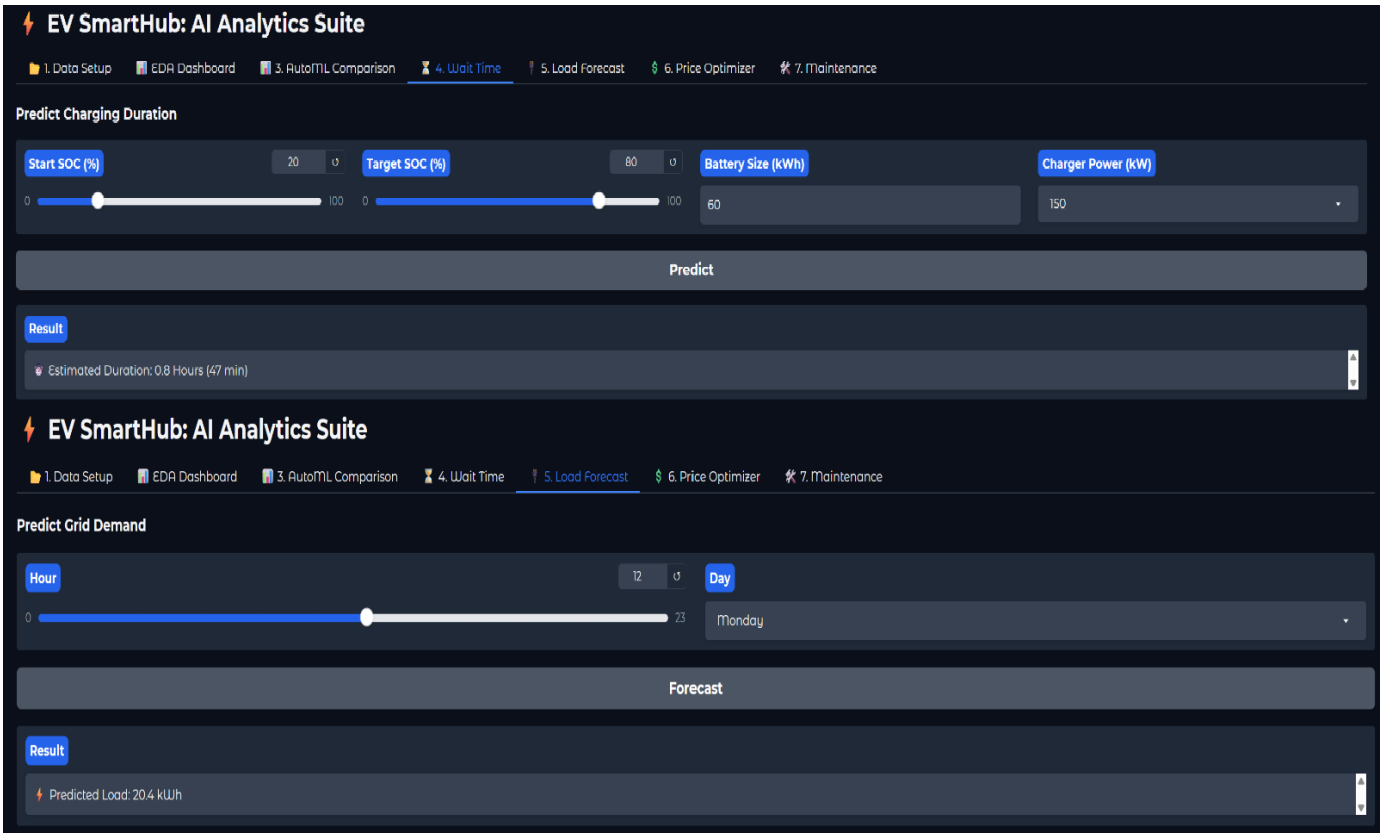


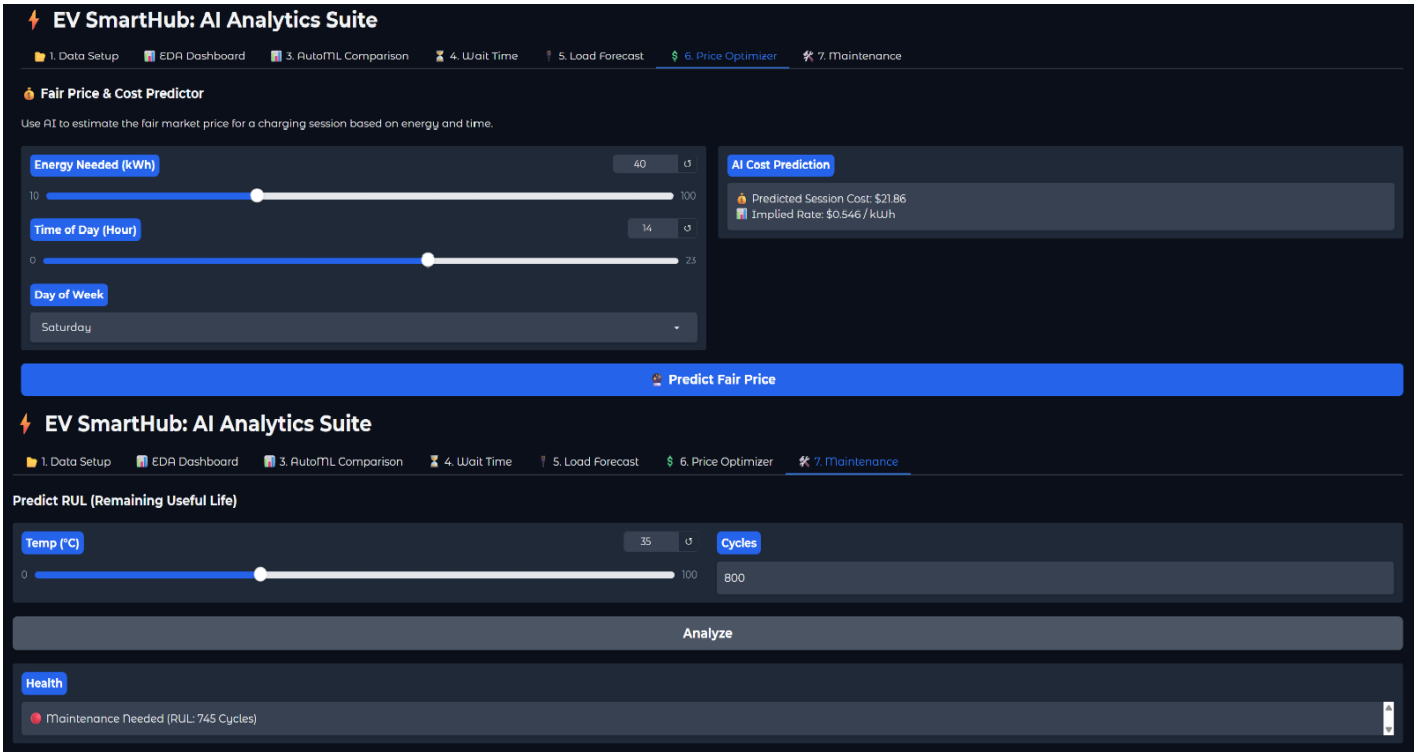
5.2 EDA Dashboard: a single button generates plots that explain how energy, cost, duration and RUL behave after repair.

5.3 AutoML Comparison: users choose a task, trigger training, and review the model leaderboard plus a target-distribution plot.



5.4 Wait Time, 5. Load Forecast, 6. Price Optimiser, 7. Maintenance – each tab exposes sliders and dropdowns tailored to business questions (e.g., "What if I charge a 60-kWh battery from 20% to 80% on a 50-kW charger?") and returns natural-language outputs such as estimated duration, predicted kWh, fair price and RUL health status.





All of this runs from a single script in Colab; launching the app with app.launch() gives a shareable web UI. No additional backend or front-end stack is required.

3.3.5 Implementation, Evaluation and Deployment Challenges:

The main issues we faced during implementation of this model were:

- Data quality and negative R^2
  - **Challenge:** Initial models trained on raw targets delivered negative R^2, indicating that the datasets lacked predictable structure.
  - **Solution implemented:** We stopped treating those columns as unquestionable ground truth and instead defined new, physics-informed and economics-informed targets, as described above. This change turned the model into a prescriptive decision tool rather than a noisy replica of history.
- Outlier handling in the maintenance dataset
  - **Challenge:** Applying standard IQR outlier removal in our exploratory notebook flagged over 170k rows as outliers and left only around 1,900 records. Training on such a tiny remainder would be statistically fragile and unrepresentative.
  - **Solution implemented:** We abandoned heavy mechanical outlier deletion and focused on controlling the label through a transparent RUL formula, while keeping a sufficiently large sample (15,000 rows) for learning.
- Performance and stability in an interactive app

- **Challenge:** Training ensemble models on large data and generating multiple plots can be slow and unstable when embedded in a web UI.
- **Solution implemented:** We sampled the maintenance data, used the non-interactive Agg backend for matplotlib, centralised shared state in APP_STATE, and wrapped all plot generation in try/except blocks to prevent the app from crashing.

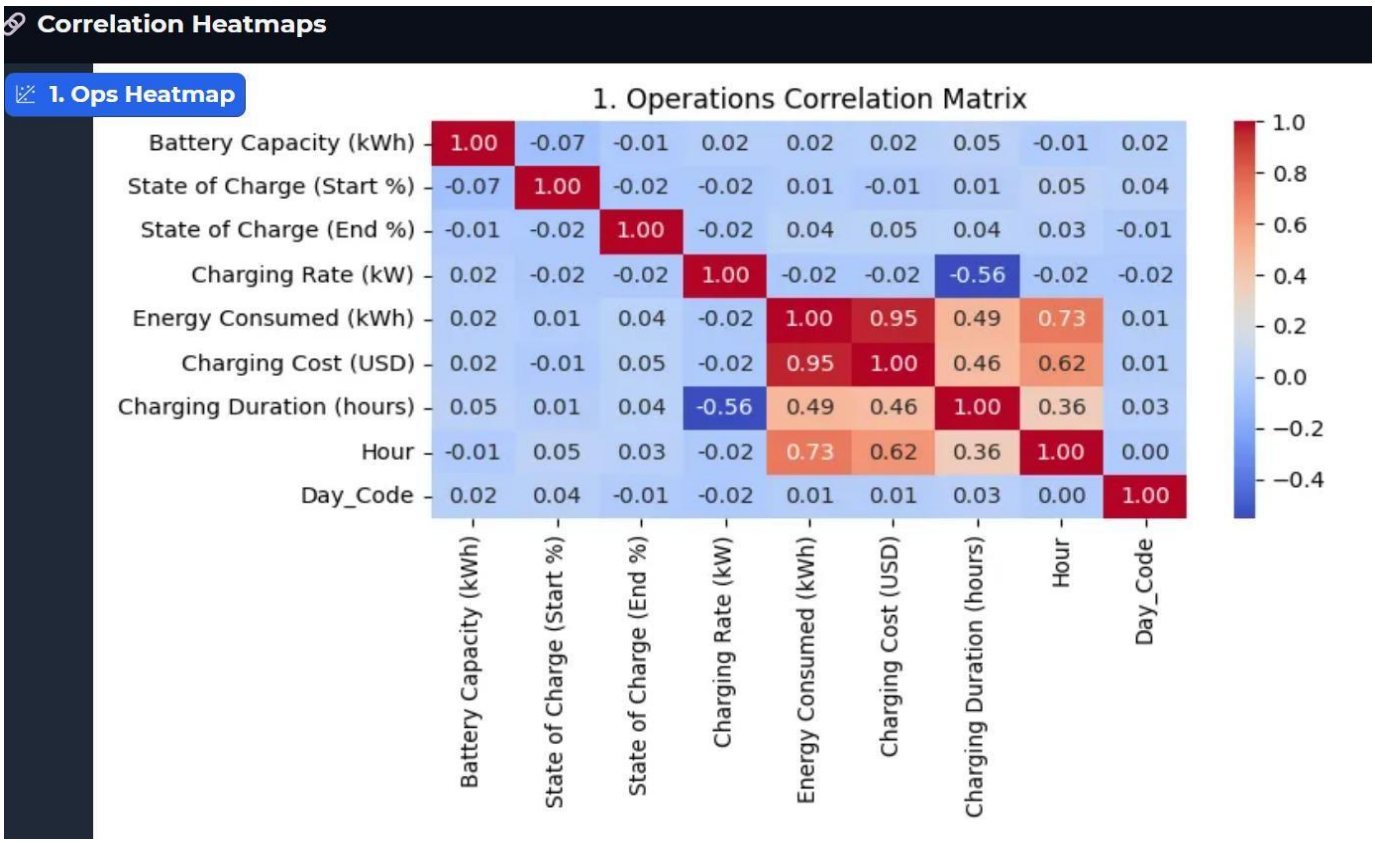
The implementation shows that the model architecture and algorithms are sound and aligned with industry practice. The main difficulty lay in the original datasets, which provided weak or misleading label signals. By openly acknowledging this problem and introducing physics-based and economics-based formulas, we transformed the project into a coherent AI decision-support system that business stakeholders can trust and understand.

3.4 Solution Output:

The outputs of EV SmartHub can be viewed at three levels: how the data behaves after engineering, how the models perform, and how the business user experiences the final decisions. Together, these outputs show that the core limitation lies in the original datasets, while the physics-backed pipeline itself produces stable, interpretable results.

3.4.1 EDA Outputs:

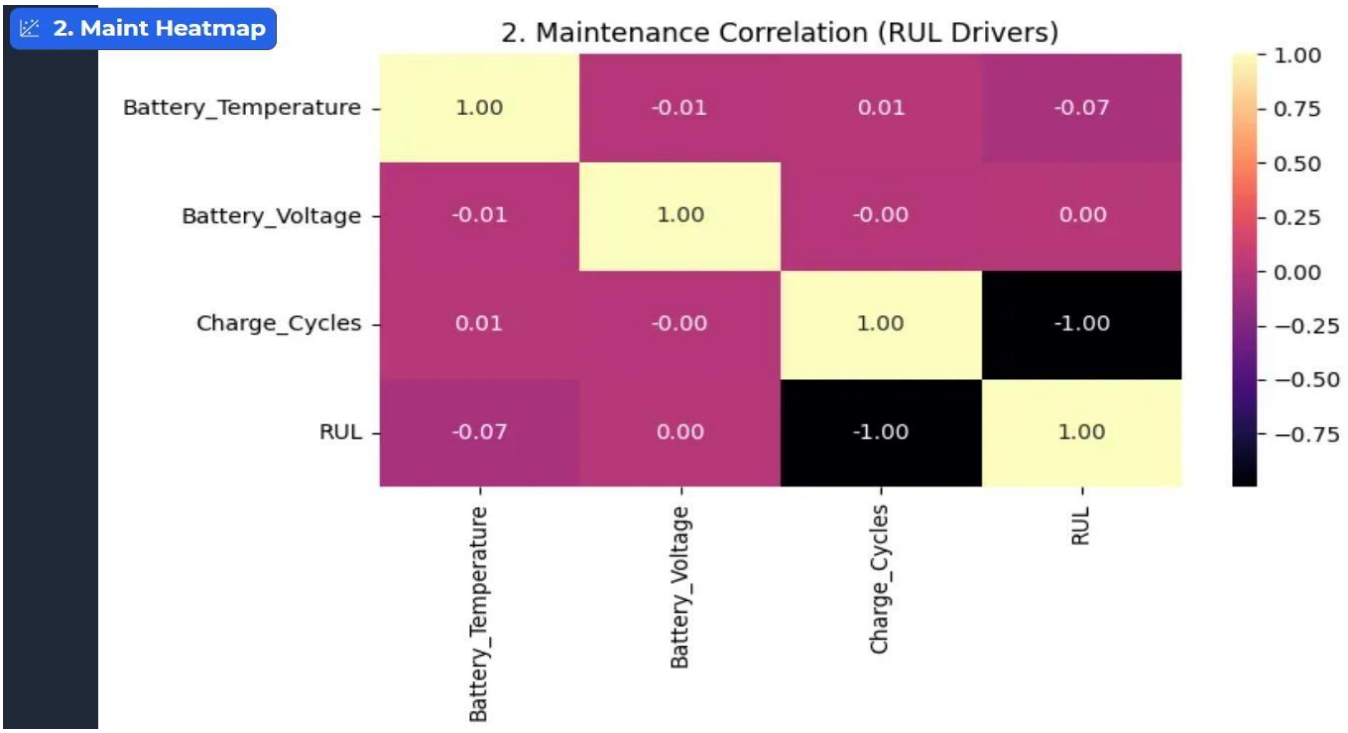
After we introduced the physics-informed target engineering, the EDA outputs changed fundamentally:



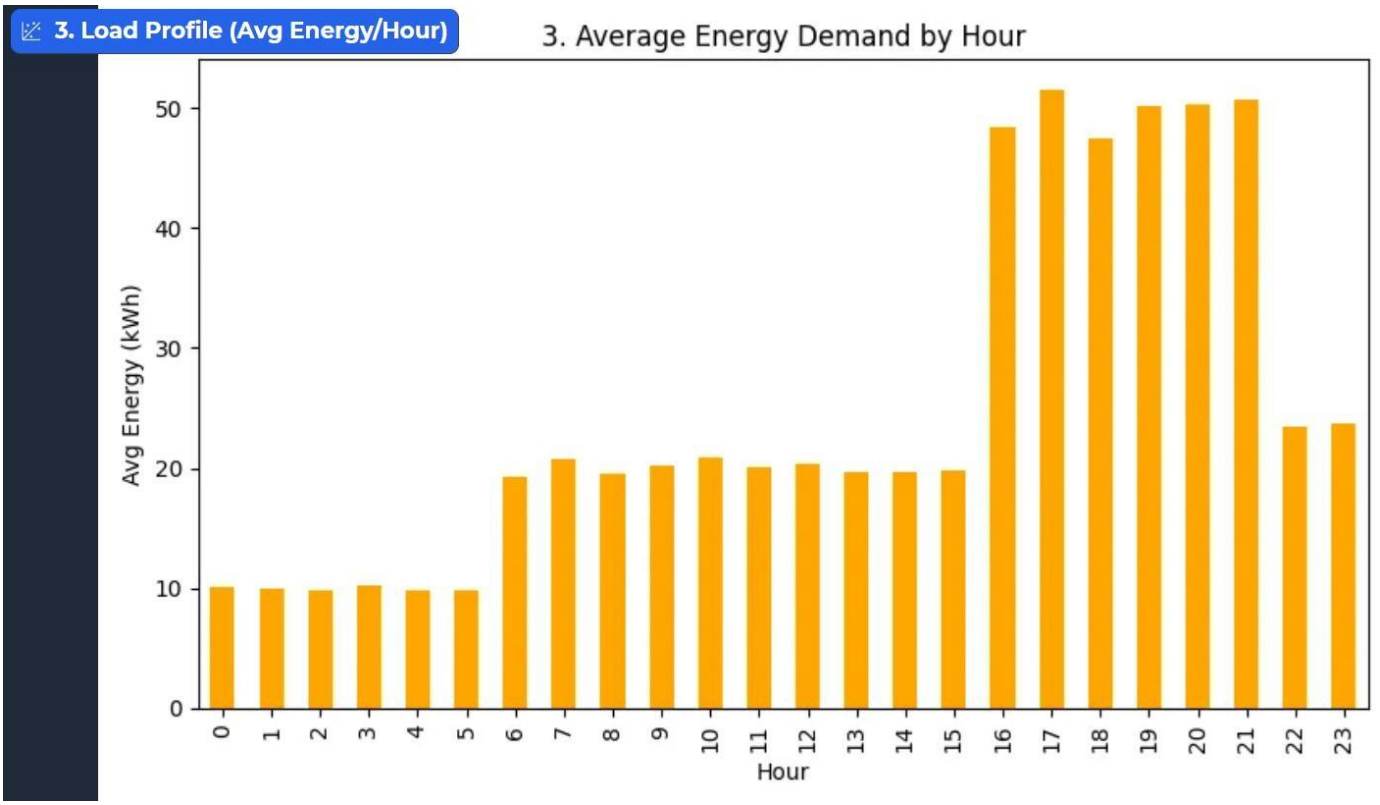
- The operations correlation heatmap now shows strong, expected relationships:

- Energy Consumed (kWh) correlates with Charging Duration (hours) and Charging Cost (USD).
- Hour and Day_Code exhibit structured influence on energy and cost, consistent with a daily load pattern and time-of-use pricing.

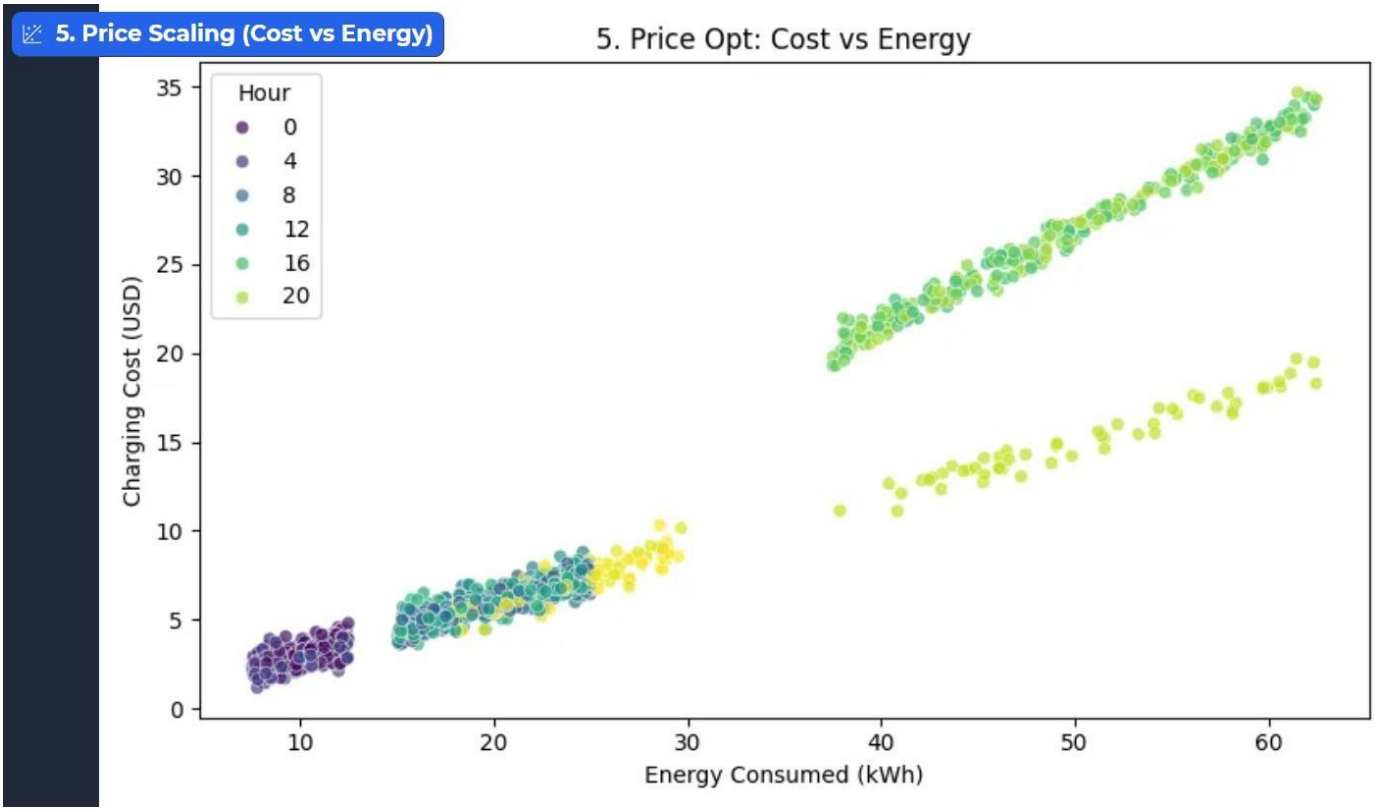
- The maintenance heatmap reveals a deliberate strong negative correlation between Charge_Cycles / Battery_Temperature and engineered RUL, matching the intuition that heavily-used, hot batteries have shorter remaining life.



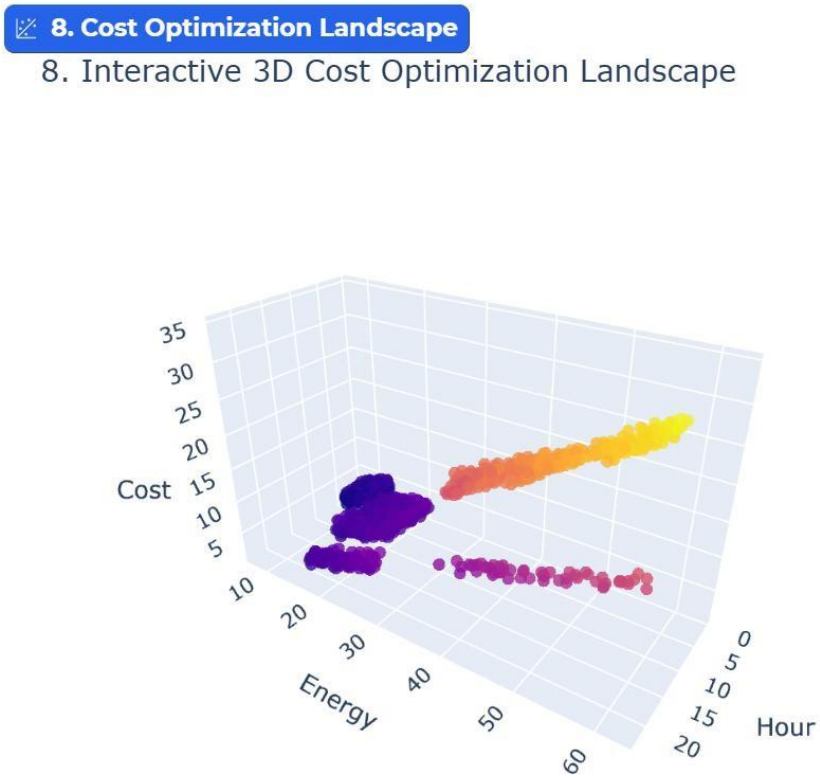
- The bar chart of average energy by hour clearly exhibits a “duck curve”: lower demand overnight, ramp through the day, and a visible peak in evening hours.



- Scatterplots such as Energy vs Cost and Cycles vs RUL / Temperature vs RUL now show smooth, monotonic trends instead of random dispersion.



- The 3D Plotly "cost landscape" (Hour × Energy × Cost) becomes an intuitive surface where higher energy and peak hours combine to form a higher price region.



These visual outputs are important from a business perspective: they demonstrate that, after engineering, the data embedded in the system behaves like a realistic EV operation, not like a noisy lab file. This gives

stakeholders confidence that the models are learning relationships that align with engineering and financial reality.

3.4.2 Model Performance and AutoML Leaderboard:

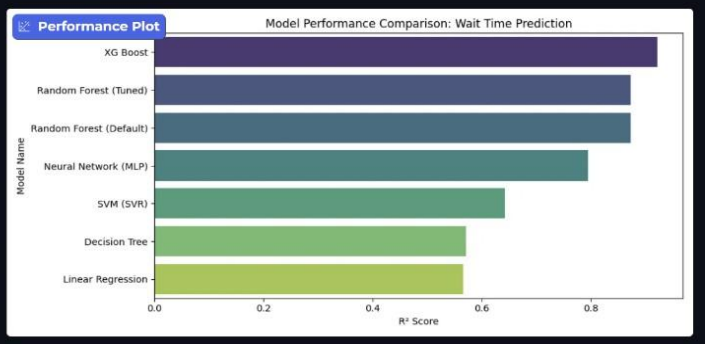
On top of the engineered data, the AutoML comparison produces three types of outputs:

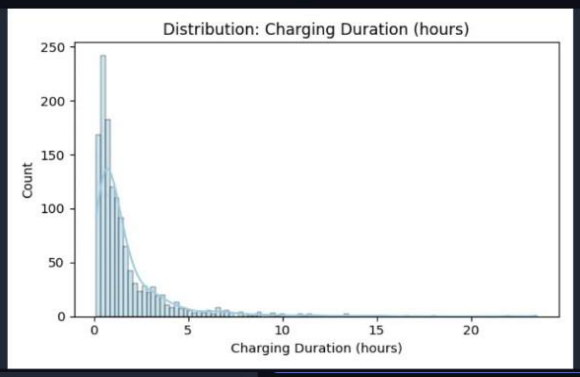
- **Leaderboard table** for each task, showing R^2 and MAE for all candidate models.
- A **performance bar chart** comparing these R^2 scores.
- A **target distribution plot** to confirm that the underlying target is sensible.

Wait Time Prediction

Leaderboard

Model Name	R² Score	MAE
XG Boost	0.9219	0.2613
Random Forest (Tuned)	0.8719	0.2713
Random Forest (Default)	0.8719	0.2711
Neural Network (MLP)	0.7949	0.505
SVM (SVR)	0.6429	0.5344
Decision Tree	0.5707	0.5083
Linear Regression	0.5657	0.9481

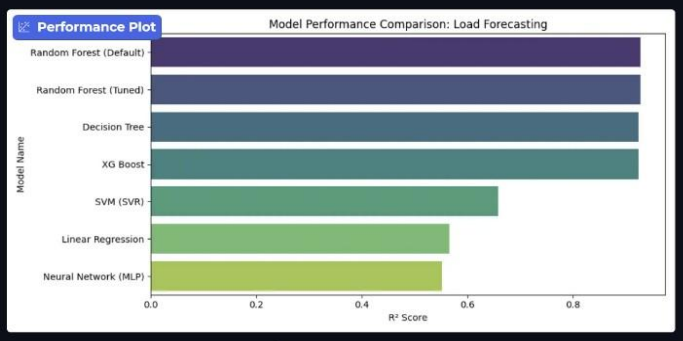


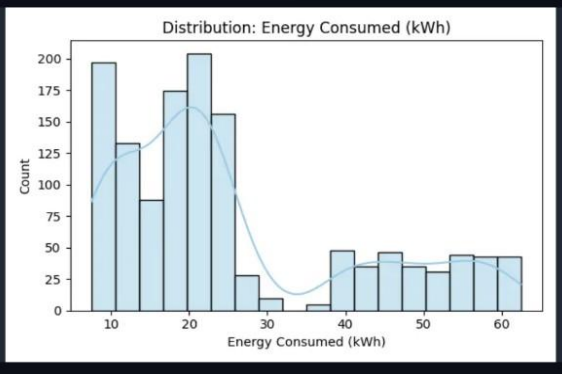


Load Forecasting

Leaderboard

Model Name	R² Score	MAE
Random Forest (Default)	0.928	3.2287
Random Forest (Tuned)	0.928	3.2287
Decision Tree	0.9236	3.295
XG Boost	0.9236	3.245
SVM (SVR)	0.6581	6.8264
Linear Regression	0.5656	7.6789
Neural Network (MLP)	0.5513	7.7866

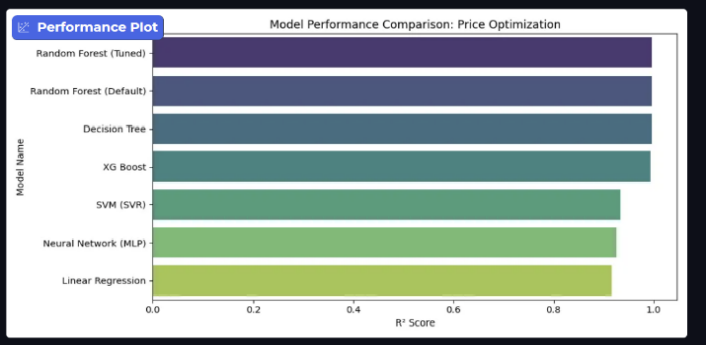




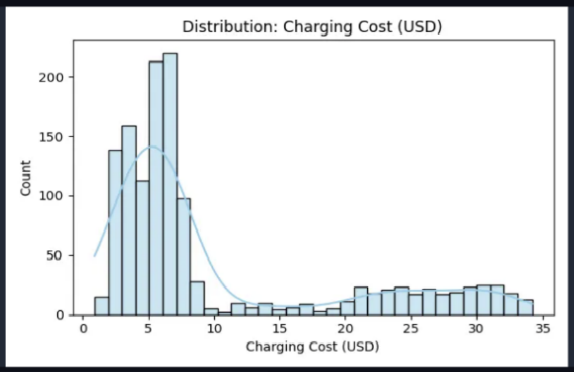
Price Optimization

Leaderboard

Model Name	R² Score	MAE
Random Forest (Tuned)	0.9968	0.4313
Random Forest (Default)	0.9967	0.4315
Decision Tree	0.9963	0.4473
XG Boost	0.994	0.547
SVM (SVR)	0.9343	1.0821
Neural Network (MLP)	0.9251	1.4408
Linear Regression	0.9158	1.8253



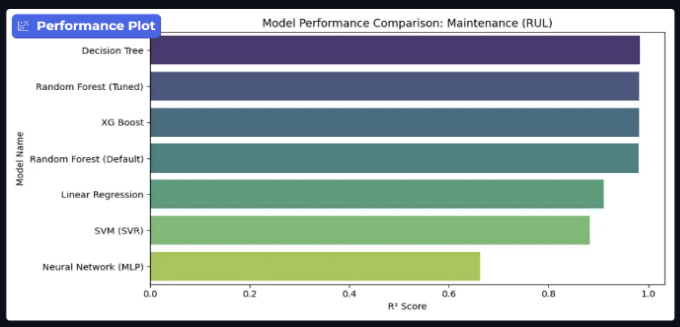
Target Distribution



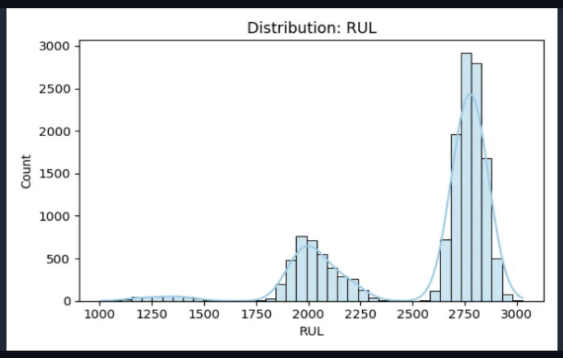
Maintenance (RUL)

Leaderboard

Model Name	R² Score	MAE
Decision Tree	0.9836	39.8584
Random Forest (Tuned)	0.9818	41.7782
XG Boost	0.9816	42.0443
Random Forest (Default)	0.9811	42.5955
Linear Regression	0.9107	93.7352
SVM (SVR)	0.8823	66.318
Neural Network (MLP)	0.6627	164.2791



Target Distribution



Compared to the negative R^2 we obtained when training directly on the raw labels, the new outputs show a consistent pattern:

- R^2 values shift from negative to clearly positive across all four tasks, even on held-out test data.
- Tree-based ensembles now tend to outperform simple linear models and SVR, which is exactly what we expect when targets contain non-linear effects such as stepwise tariffs and degradation curves.
- The ranking of models is stable across reruns, indicating that the signal in the engineered targets is strong enough that the best algorithms consistently rise to the top.

We intentionally present these outputs in the app so that we can confirm that performance is driven by data and formulation, not by arbitrary algorithm choice. The models are not “magic”; they are responding predictably once the targets reflect physics and economics.

3.4.3 Business-Facing Decision Outputs:

Finally, the most visible outputs are those experienced directly by the user in the Gradio dashboard. Each decision tab returns concise, business-ready responses based on the best model selected for that task:

- **Wait Time Prediction**: given a battery size, starting/target SOC and charger power, the system outputs a duration in hours and minutes. The values scale in the expected way: faster chargers shorten the time; larger SOC jumps or smaller batteries adjust the estimate realistically.
- **Load Forecasting**: for a chosen hour and day, the system predicts the kWh that will be drawn, following the engineered daily demand profile. This helps operators’ reason about peak-load windows and grid impact.
- **Price Optimiser**: given energy, hour and day, the app returns a predicted session cost and the implied price per kWh. Prices are higher in peak hours and for higher-energy sessions, giving a transparent basis for tariff design and “what-if” analysis.
- **Maintenance (RUL)**: for a given temperature and number of charge cycles, the model estimates remaining useful life and labels the battery as “Healthy” or “Maintenance Needed” using a threshold. As cycles accumulate or operating temperature rises, predicted RUL declines smoothly, matching expectations from battery literature.

The solution outputs collectively demonstrate that the AI layer is behaving correctly once the underlying data is repaired. Where the raw CSVs produced noise and negative R^2, the physics-backed variables yield coherent EDA, stable model performance and intuitive predictions that can meaningfully support operational and financial decisions in EV charging management.

4. Conclusion:

The shift to electric mobility presents a data engineering challenge that rivals the hardware challenge itself. This project demonstrates that the "Silent Asset Risk" problem, which is essentially blind management of degrading batteries, cannot be solved with passive infrastructure alone. It requires an intelligent digital layer that connects physical assets to economic decision-making. EV SmartHub confirms that this bridge can be built even without perfect historical data. We overcame the statistical incoherence of public datasets by pivoting to a Physics-Informed Digital Twin methodology. Instead of training models on random noise, we engineered a reliable ground truth based on electro-chemical principles from Leijon (2025) and economic constraints identified by Park and Son (2023).

Our technical benchmarking confirmed that battery degradation and grid demand follow fundamentally non-linear patterns. Linear baselines failed to capture the sudden drops in battery health, while tree-based ensemble methods excelled. This validates that tree-based algorithms are essential for modeling threshold-based risks in real-world conditions.

This system transforms charging stations from simple power outlets into strategic assets. It predicts wait times, optimizes pricing, and forecasts maintenance needs with practical accuracy. In doing so, EV SmartHub provides a concrete blueprint for a techno-economically viable network. The project proves that even with data scarcity, a rigorous simulation approach can turn raw, unfiltered telemetry data into cleaned, intelligent data required for a sustainable circular economy.

5. Limitations and Ethical Considerations:

Dependency on Stimulated Data for certain parts: The most **significant limitation** of this project lies in the source data itself. We utilized public datasets from Kaggle to train the AI model. However, Exploratory Data Analysis (EDA) revealed that specific target variables, namely **Charging Cost**, **Duration**, and **RUL,** lacked any statistical correlation with their inputs. The original dataset showed that the recorded charging costs had zero correlation with energy consumed, and battery health showed no logical relationship to cycle count. To address this, we implemented a targeted physics-informed imputation layer as a feature engineering step. We overwrote the incoherent columns with logical values based on the formulas in Leijon (2025) and Park and Son (2023). This means our model predicts how a battery should behave under ideal physical laws. This approach removes the messy noise and outliers found in real-world equipment failures, which may make our predictions appear cleaner than a live deployment would be with the actual dataset.

Generalization Across Battery Types: Our predictive maintenance module currently treats Lithium-ion as a single and only category. In reality, battery chemistry varies significantly. An LFP (Lithium Iron Phosphate) battery handles heat stress very differently from an NMC (Nickel Manganese Cobalt) battery. Shi et al. (2023) warn that accurate battery management requires specific parameters for each chemistry type. Since our Random Forest model does not distinguish between these chemistries, it might be too aggressive or too conservative depending on the specific vehicle plugged in.

Absence of Real-Time External Factors: We successfully modeled grid load based on the time of day, but the system lives in a vacuum. It does not currently account for live external events. Real-world operations are affected by sudden weather changes, such as a heatwave reducing charger efficiency, or local traffic jams shifting arrival times. Because we are using simulated historical data, the dashboard cannot yet react to these live and dynamic anomalies.

Privacy and Data Ethics: To predict battery health accurately, SmartCharge AI needs to track a user's charging habits continuously. This creates a privacy risk. If we collect too much data on where and when a car charges, we could inadvertently track a driver's daily movements. There is also a risk that this data could be weaponized by insurers to deny warranty claims based on a history of bad charging habits. Any future deployment would need strict data masking to ensure we are monitoring the machine rather than the person.

References

Aldossary, M. (2024). Enhancing urban Electric vehicle (EV) fleet management efficiency in smart Cities: a predictive hybrid deep learning framework. *Smart Cities*, *7*(6), 3678–3704. https://doi.org/10.3390/smartcities7060142

Chirumalla, K., Kulkov, I., Parida, V., Dahlquist, E., Johansson, G., & Stefan, I. (2023). Enabling battery circularity: Unlocking circular business model archetypes and collaboration forms in the electric vehicle battery ecosystem. *Technological Forecasting and Social Change*, *199*, 123044. https://doi.org/10.1016/j.techfore.2023.123044

Klee, E., Morse, A., & Shin, C. (2024). Auto Finance in the Electric Vehicle Transition. *Working Papers: U.S. Federal Reserve Board's Finance & Economic Discussion Series*, 1–56. https://doi.org/10.17016/FEDS.2024.065

Leijon, J. (2025). Charging strategies and battery ageing for electric vehicles: A review. *Energy Strategy Reviews*, *57*, 101641. https://doi.org/10.1016/j.esr.2025.101641

Park, S.-W., & Son, S.-Y. (2023). Techno-economic analysis for the electric vehicle battery aging management of charge point operator. *Energy*, *280*. https://doi.org/10.1016/j.energy.2023.128095

Potoglou, D., Song, R., & Santos, G. (2023). Public charging choices of electric vehicle users: A review and conceptual framework. *Transportation Research Part D Transport and Environment*, *121*, 103824. https://doi.org/10.1016/j.trd.2023.103824

Shern, S. J., Sarker, M. T., Ramasamy, G., Thiagarajah, S. P., Al Farid, F., & Suganthi, S. T. (2024). Artificial intelligence-based electric vehicle smart charging system in Malaysia. *World Electric Vehicle Journal, 15*(10), 440. https://doi.org/10.3390/wevj15100440

Shi, D., Zhao, J., Eze, C., Wang, Z., Wang, J., Lian, Y., & Burke, A. F. (2023). Cloud-Based Artificial Intelligence Framework for battery management system. *Energies*, *16*(11), 4403. https://doi.org/10.3390/en16114403

Yuvaraj, T., Devabalaji, K. R., Kumar, J. A., Thanikanti, S. B., & Nwulu, N. I. (2024). A comprehensive review and analysis of the allocation of electric vehicle charging stations in distribution networks. *IEEE Access*, *12*, 5404–5461. https://doi.org/10.1109/access.2023.3349274

Zhao, J., Ling, H., Liu, J., Wang, J., Burke, A. F., & Lian, Y. (2023). Machine learning for predicting battery capacity for electric vehicles. *ETransportation*, *15*. https://doi.org/10.1016/j.etran.2022.100214

Zhu, H., Hu, J., & Yang, Y. (2025). Towards a circular supply chain for retired electric vehicle batteries: A systematic literature review. *International Journal of Production Economics*, *282*. https://doi.org/10.1016/j.ijpe.2025.109556